

Ejercicio de Examen – Función con CTE (MTVF)

Tema: Funciones con retorno de tabla, CTE, validaciones y uso correcto de SQL Server.

Enunciado

La empresa **Charming Luxury Spa** necesita analizar la ocupación histórica de sus habitaciones para optimizar tarifas y disponibilidad. Tenés la tabla:

Reservas

ID_Reserva	ID_Habitacion	FechaIngreso	FechaEgreso	Estado
------------	---------------	--------------	-------------	--------

int	int	date	date	varchar(20)
-----	-----	------	------	-------------

Reglas del negocio:

- Una reserva puede estar en estado: 'Confirmada', 'Check-In', 'Check-Out', 'Cancelada'.
- El rango [FechaIngreso, FechaEgreso) debe interpretarse como **ingreso inclusive, egreso exclusivo**.

Consigna

Crear una **función con valor de tabla (MTVF)** llamada:

fn_OcupacionPorHabitacion

La función debe recibir:

(@ID_Habitacion INT, @FechaDesde DATE, @FechaHasta DATE)

y debe **devolver un conjunto de registros**, uno por cada día del rango solicitado, con la siguiente estructura:

Fecha	TieneReserva
-------	--------------

date	bit
------	-----

Requisitos obligatorios

1. **Debe usarse al menos una CTE** para generar el conjunto de fechas “día por día”.
 - No se puede usar una tabla auxiliar del sistema.
 - No se puede usar WHILE ni cursores (la MTVF no lo permite).
2. Validar que:
 - @FechaDesde < @FechaHasta, si no → devolver tabla vacía.
(Hint: esta validación también debe resolverse dentro de la CTE o mediante un filtro en la MTVF).
3. Una fecha tendrá TieneReserva = 1 únicamente si **existe una reserva válida** para esa fecha:
 - Que coincida con la habitación.

- Que se superponga con el día analizado.
 - Que no esté cancelada.
4. El resultado debe venir **ordenado por Fecha ascendente**.

Pista conceptual (sin resolver el ejercicio)

Para completar este ejercicio, vas a necesitar:

- Una **CTE recursiva** que construya un calendario día a día.
- Un **LEFT JOIN** entre ese calendario y la tabla de Reservas.
- Una expresión CASE que determine si hay solapamiento entre la fecha del calendario y el rango de la reserva.
- Filtros para estados válidos: Estado IN ('Confirmada','Check-In','Check-Out').
- Una MTVF porque una función escalar no permite devolver varios registros ni usar RETURN SELECT.

SOLUCIÓN MODELO – fn_OcupacionPorHabitacion

1. Eliminación previa

```
IF OBJECT_ID('dbo.fn_OcupacionPorHabitacion', 'IF') IS NOT NULL
    DROP FUNCTION dbo.fn_OcupacionPorHabitacion;
GO
```

2. Creación de la función (MTVF con CTE recursiva)

```
CREATE FUNCTION dbo.fn_OcupacionPorHabitacion
(
    @ID_Habitacion INT,
    @FechaDesde DATE,
    @FechaHasta DATE
)
RETURNS TABLE
AS
RETURN
(
    -----
    -- VALIDACIÓN CONDICIÓN DE RANGO
    -- Si el rango no es válido, la CTE simplemente no generará filas
    -----
    WITH CTE_Calendarario AS
    (
        -- Primer día del rango
        SELECT @FechaDesde AS Fecha
        WHERE @FechaDesde < @FechaHasta

        UNION ALL

        -- Construcción recursiva día por día
        SELECT DATEADD(DAY, 1, Fecha)
```

```

        FROM CTE_Calendarario
        WHERE DATEADD(DAY, 1, Fecha) < @FechaHasta
    ),

    -----
    -- CTE que determina si cada día tiene una reserva válida
    -----
    CTE_Ocupacion AS
    (
        SELECT
            C.Fecha,
            CASE
                WHEN EXISTS (
                    SELECT 1
                    FROM Reservas R
                    WHERE R.ID_Habitacion = @ID_Habitacion
                        AND R.Estado IN ('Confirmada', 'Check-In', 'Check-Out')
                        AND C.Fecha >= R.FechaIngreso
                        AND C.Fecha < R.FechaEgreso -- egreso exclusivo
                )
                THEN 1
                ELSE 0
            END AS TieneReserva
        FROM CTE_Calendarario AS C
    )

    -----
    -- Resultado final ordenado
    -----
    SELECT Fecha, TieneReserva
    FROM CTE_Ocupacion
    ORDER BY Fecha ASC
);
GO

```

-- Habilitar la recursión profunda si es necesario (hasta 100 días alcanza sin esto)

-- OPTION (MAXRECURSION 0);

3. Ejemplo de uso

```
SELECT *
```

```
FROM dbo.fn_OcupacionPorHabitacion(101, '2025-01-01', '2025-01-15');
```

Obligación cumplida: Uso de CTE recursiva

- Se genera un calendario desde @FechaDesde hasta @FechaHasta - 1.
- MTVF no permite WHILE, así que la única forma correcta es con una **CTE recursiva**.

Validación del rango

- Si @FechaDesde >= @FechaHasta, la primera parte de la CTE no genera ninguna fila → función retorna vacío sin error.

Detección de solapamiento

Una fecha está reservada si se cumple:

Fecha $\in$ [FechaIngreso ; FechaEgreso)

Esto se implementa con:

C.Fecha $\geq$ R.FechaIngreso

AND C.Fecha < R.FechaEgreso

Evitar reservas canceladas

Estado IN ('Confirmada','Check-In','Check-Out')

Es MTVF, devuelve conjunto y soporta lógica no declarativa mínima

- No usa variables internas.
- No usa RETURN escalar.
- Cumple todas las restricciones del examen.

Es la **única manera en SQL Server** de generar filas repetitivas dentro de una función MTVF, porque:

- no podés usar WHILE
- no podés usar INSERT
- no podés usar tablas temporales
- no podés usar variables de tabla dentro de la función

La CTE recursiva te reemplaza todo eso.

Una CTE recursiva tiene esta estructura:

CTE = caso base

UNION ALL

CTE = transformación del CTE anterior

Función ITVF con condición y JOIN

Enunciado

Crear una función **Inline Table-Valued Function (ITVF)** llamada fn_VentasPorCliente que reciba un @ClienteID y devuelva:

- ID_Cliente
- Nombre
- TotalFacturado

Debe sumar el importe de todas las facturas del cliente.

Debe ser *inline*, sin BEGIN/END.

Solución

```
CREATE OR ALTER FUNCTION fn_VentasPorCliente (@ClienteID INT)
RETURNS TABLE
AS
RETURN
(
    SELECT
        c.ID_Cliente,
        c.Nombre,
        SUM(f.Importe) AS TotalFacturado
    FROM Clientes c
    INNER JOIN Facturas f ON c.ID_Cliente = f.ID_Cliente
    WHERE c.ID_Cliente = @ClienteID
    GROUP BY c.ID_Cliente, c.Nombre
);
```

Subconsulta anidada con promedio

Enunciado

Listar los productos cuyo **precio** está por encima del **precio promedio global** de todos los productos.

Solución

```
SELECT
    p.ID_Producto,
    p.Nombre,
    p.Precio
FROM Productos p
WHERE p.Precio > (
    SELECT AVG(Precio) FROM Productos
);
```

Vista con JOIN + filtro

Enunciado

Crear una vista vw_StockBajo que muestre:

- ID_Producto
- Nombre
- Stock
- Categoría

Solo debe mostrar productos con stock menor a 20 unidades.

Solución

```
CREATE OR ALTER VIEW vw_StockBajo
AS
```

```

SELECT
    p.ID_Producto,
    p.Nombre,
    p.Stock,
    c.NombreCategoria
FROM Productos p
INNER JOIN Categorías c ON p.ID_Categoría = c.ID_Categoría
WHERE p.Stock < 20;

```

Procedimiento con transacción y manejo de errores

Enunciado

Crear el procedimiento sp_RegistrarPago, que:

- Inserte un pago en la tabla Pagos.
- Actualice la deuda del cliente en la tabla Clientes.
- Use transacción + TRY CATCH.
- Devuelva un mensaje de éxito o error.

Solución

```

CREATE OR ALTER PROCEDURE sp_RegistrarPago
    @ID_Cliente INT,
    @Importe DECIMAL(10,2)
AS
BEGIN TRY
    BEGIN TRAN;

    INSERT INTO Pagos (ID_Cliente, Importe, Fecha)
    VALUES (@ID_Cliente, @Importe, GETDATE());

    UPDATE Clientes
    SET Deuda = Deuda - @Importe
    WHERE ID_Cliente = @ID_Cliente;

    COMMIT TRAN;
    SELECT 'Pago registrado correctamente' AS Mensaje;
END TRY
BEGIN CATCH
    ROLLBACK TRAN;
    SELECT 'Error al registrar el pago: ' + ERROR_MESSAGE() AS Mensaje;
END CATCH

```

Productos sin ventas (subconsulta NOT EXISTS)

Enunciado

Obtener todos los productos que **no tienen ninguna venta registrada** en la tabla Ventas.

Solución

```

SELECT
    p.ID_Producto,
    p.Nombre
FROM Productos p
WHERE NOT EXISTS (
    SELECT 1
    FROM Ventas v
    WHERE v.ID_Producto = p.ID_Producto
);

```

Evitar lecturas sucias con nivel de aislamiento

Enunciado

Crear un procedimiento sp_TotalVentasSeguro que:

- Calcule el total de ventas de la tabla Ventas.
- Evite lecturas sucias (dirty reads) usando un nivel de aislamiento adecuado.
- Devuelva un mensaje indicando el total.

No se permite usar READ UNCOMMITTED.

Solución

```
CREATE OR ALTER PROCEDURE sp_TotalVentasSeguro
AS
BEGIN
    SET NOCOUNT ON;

    -- Evita lecturas sucias
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    DECLARE @Total DECIMAL(18,2);

    SELECT @Total = SUM(Importe)
    FROM Ventas;

    SELECT 'Total de ventas (sin lecturas sucias): ' + CAST(@Total AS
    VARCHAR(50)) AS Mensaje;
END
```

CTE recursivo: cadena jerárquica

Enunciado

Dada la tabla Empleados (ID, Nombre, ID_Jefe), crear una consulta con CTE recursiva que:

- Devuelva todos los empleados
- Incluya su nivel dentro de la jerarquía (1 = jefe máximo)

Solución

```
WITH Jerarquia AS (
    SELECT
        ID,
        Nombre,
        ID_Jefe,
        1 AS Nivel
    FROM Empleados
    WHERE ID_Jefe IS NULL

    UNION ALL

    SELECT
        e.ID,
        e.Nombre,
        e.ID_Jefe,
        j.Nivel + 1
    FROM Empleados e
    INNER JOIN Jerarquia j ON e.ID_Jefe = j.ID
)
SELECT *
FROM Jerarquia
ORDER BY Nivel, Nombre;
```

MTVF con bucle WHILE para calcular totales

Enunciado

Crear una **Multi-Statement TVF** fn_TopClientesPorFacturacion que:

- Recorra la tabla Clientes con un WHILE
- Sume la facturación de cada cliente
- Devuelva solo los clientes cuyo total supere un valor mínimo pasado por parámetro

Solución

```
CREATE OR ALTER FUNCTION fn_TopClientesPorFacturacion (@Minimo DECIMAL(18,2))
RETURNS @Resultado TABLE (
    ID_Cliente INT,
    Nombre NVARCHAR(100),
    TotalFacturado DECIMAL(18,2)
)
AS
BEGIN
    DECLARE @ID INT = 1;
    DECLARE @MaxID INT = (SELECT MAX(ID_Cliente) FROM Clientes);

    WHILE @ID <= @MaxID
    BEGIN
        IF EXISTS (SELECT 1 FROM Clientes WHERE ID_Cliente = @ID)
        BEGIN
            DECLARE @Total DECIMAL(18,2);

            SELECT @Total = SUM(Importe)
            FROM Facturas
            WHERE ID_Cliente = @ID;

            IF @Total >= @Minimo
            BEGIN
                INSERT INTO @Resultado
                SELECT ID_Cliente, Nombre, @Total
                FROM Clientes
                WHERE ID_Cliente = @ID;
            END
        END

        SET @ID += 1;
    END

    RETURN;
END
```

EJERCICIO 9 — Transacción completa con validación + rollback + mensaje custom

Enunciado

Crear un procedimiento sp_TransferirSaldo que:

- Reciba @Origen, @Destino, @Monto
- Verifique que el origen tenga saldo suficiente
- Si no lo tiene → RAISERROR con mensaje custom
- Si lo tiene → debita e acredita
- Todos los pasos deben ir dentro de una transacción con ROLLBACK en caso de error

Solución

```
CREATE OR ALTER PROCEDURE sp_TransferirSaldo
    @Origen INT,
    @Destino INT,
    @Monto DECIMAL(18,2)
AS
BEGIN TRY
    BEGIN TRAN;

    DECLARE @SaldoOrigen DECIMAL(18,2);

    SELECT @SaldoOrigen = Saldo FROM Cuentas WHERE ID_Cuenta = @Origen;

    IF @SaldoOrigen < @Monto
        RAISERROR('Saldo insuficiente en la cuenta origen.', 16, 1);

    UPDATE Cuentas SET Saldo = Saldo - @Monto WHERE ID_Cuenta = @Origen;
    UPDATE Cuentas SET Saldo = Saldo + @Monto WHERE ID_Cuenta = @Destino;

    COMMIT TRAN;
    SELECT 'Transferencia realizada correctamente' AS Mensaje;
END TRY
BEGIN CATCH
    ROLLBACK TRAN;
    SELECT 'Error: ' + ERROR_MESSAGE() AS Mensaje;
END CATCH
```

EJERCICIO 10 — Evitar lecturas fantasmas con SERIALIZABLE

Enunciado

Crear un procedimiento sp_ValidarStockUnico que:

- Inicie transacción con SERIALIZABLE
- Verifique si existe un producto con cierto nombre
- Si no existe → lo crea
- Si existe → retorna mensaje
- Debe evitar **lecturas fantasmas** (dos transacciones creando el mismo producto)

Solución

```
CREATE OR ALTER PROCEDURE sp_ValidarStockUnico
    @NombreProducto VARCHAR(100),
    @Precio DECIMAL(10,2)
AS
BEGIN TRY
    SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
    BEGIN TRAN;

    IF EXISTS (SELECT 1 FROM Productos WHERE Nombre = @NombreProducto)
    BEGIN
        ROLLBACK TRAN;
        SELECT 'El producto ya existe' AS Mensaje;
        RETURN;
    END

    INSERT INTO Productos (Nombre, Precio)
    VALUES (@NombreProducto, @Precio);

    COMMIT TRAN;
    SELECT 'Producto creado correctamente' AS Mensaje;
END TRY
BEGIN CATCH
    ROLLBACK TRAN;
```

```
SELECT 'Error: ' + ERROR_MESSAGE() AS Mensaje;  
END CATCH
```